```
package com.Abdul.OOP;

public class WrapperClass {
    public static void main(String[] args) {
        // Primitive data types are stored in Stack.
        int a = 10;
        int b = 20;

        // To use primitive data types in heap use wrapper class
        // It will store data type as object
        Integer num = 45;
        // Here 45 is created as an object in heap.

        swap(a, b);
        System.out.println(a + " " + b);

        Integer num1 = 10;
        Integer num2 = 20;
        swap2(num1, num2);
        System.out.println(num1 + " " + num2);


 *********
        /*
        For primitive data types values cannot be reassigned.
 For example:
       final int a = 10;
       You cannot do something like a = 20;

        For non-primitive data types you can make changes in values but cannot reassign it means if you
create an object then
 that reference variable will always point to the same object which it is pointing while creation.
 It can never point to other object although object's instance variable value can change.
        For eg, if you have Student abdul = new Student();
        then, you can do abdul.name = "Rehman";
        but cannot do abdul = other object
      */



        final A abdul = new A("Abdul Rehman");
        // Cannot do it because of the final keyword
//          abdul = new Student("new name");

        A obj;

        for(int i = 0; i < 1000000; i++) {
            obj = new A("Random name");
        }
    }

    // Numbers will not be swapped because here the value is passed i.e. 10 and 20 are passed
    static void swap(int a, int b) {
        int temp = a;
```

```
        a = b;
        b = temp;
    }


    // If we do something like this then also the values will not be swapped because
    // Integer wrapper class is final i.e. we cannot modify the value which are final
    // For eg.
    final int increase = 2;

    // It will throw an error because final values cannot be reassigned
    //    increase = 3;
    static void swap2(int a, int b) {
        int temp = a;
        a = b;
        b = temp;
    }

}

class A {
    final int num = 10;
    String name;

    A(String name) {
        this.name = name;
    }


    // We cannot destroy object in java manually as in C++
    // But we can make something do when it is automatically deleted by java compiler
    protected void finalize() throws Throwable {
        System.out.println("Object is destroyed");
    }
}
```